

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

Ingegneria del Software 2

Analisi di code smell, defect prediction e refactoring automatico in Apache Storm

**Report finale - percorso Prof. Davide
Falessi**

Studente	Valerio Torac
Progetto	Apache Storm
Baseline	Apache Storm 3.0.0 (tag v3.0.0)
Repository	BestOfTorac/storm - branch isw2-project
Anno accademico	2025/2026

Settembre 2026

Indice

Sezioni

1. Introduzione	3
2. Metodologia	3
2.1 Milestone 1 - Dataset creation	3
2.2 Milestone 2 - Classifier accuracy	6
2.3 Milestone 3 - What-if analysis	6
2.4 Milestone 4 - Automated refactoring	6
3. Risultati	7
3.1 Milestone 1 - Caratterizzazione del dataset	7
3.2 Milestone 2 - Accuratezza dei classificatori	9
3.3 Milestone 3 - Bug potenzialmente prevenibili	10
3.4 Milestone 4 - Esito del refactoring automatico	11
4. Threats to validity	12
5. Conclusioni	14
5.1 Impatto per i practitioner	14
5.2 Impatto per i researcher	15
Riferimenti e tracciabilità	15

Tabelle e figure

Tabella 1 - Feature utilizzate nel Dataset A	4
Tabella 2 - Classi production e osservazioni BUGGY per release	7
Tabella 3 - Risultati medi della validazione M2	9
Tabella 4 - Dataset e predizioni dell'analisi what-if	10
Tabella 5 - Risultati M4 per UIHelpers	11
Tabella 6 - Risultati M4 per RedisFilterBolt	12
Tabella 7 - Threats to validity, conseguenze e mitigazioni	13
Tabella 8 - Artefatti canonici usati per il report	16
Figura 1 - Workflow di ricostruzione dell'evidenza BUGGY: SZZ e stima di IV	5
Figura 2 - Numero di classi production per release	8
Figura 3 - Incidenza delle osservazioni BUGGY per release	9

1. Introduzione

In un progetto software maturo, ridurre il rischio di defect non significa soltanto correggere gli errori già emersi: significa capire quali caratteristiche del codice e della sua storia anticipano la fragilità, quali di esse possono essere modificate e se un intervento automatico produce davvero un miglioramento senza compromettere il comportamento. Questo lavoro affronta il problema come una catena sperimentale unica: dalla storia di Apache Storm si costruisce un dataset a livello di classe e release, da quel dataset si ricavano modelli di defect prediction, quindi si misura l'effetto di uno scenario senza code smell e, infine, si porta l'analisi sul codice reale attraverso refactoring automatici sottoposti a verifiche strutturali e comportamentali.

Il caso di studio è Apache Storm, piattaforma distribuita per l'elaborazione di stream in tempo reale, scelta perché combina una lunga storia evolutiva, numerose release, un issue tracker ricco e un codebase sufficientemente ampio da rendere significative sia l'analisi storica sia la sperimentazione su classi concrete. La baseline di riferimento per le analisi finali e per la selezione delle classi della Milestone 4 è Apache Storm 3.0.0, tag Git v3.0.0. Gli artefatti accademici sono mantenuti nel branch isw2-project e separati dal codice upstream nella directory isw2/, così che il case study rimanga riproducibile senza alterare la storia originale del progetto.

L'interesse dello studio sta nel collegare quattro livelli che spesso vengono osservati separatamente. La prima fase ricostruisce la defect-proneness storica delle classi; la seconda verifica quanto essa sia prevedibile mediante metriche di prodotto e di processo; la terza isola il contributo operativo degli smell in uno scenario controfattuale; la quarta verifica cosa accade quando la riduzione degli smell viene affidata a un refactoring automatico reale. In questo modo il progetto non si limita a cercare correlazioni, ma segue la stessa ipotesi dalla misura storica fino alla trasformazione del codice, mettendo in evidenza sia i benefici sia i punti in cui i proxy di qualità smettono di essere sufficienti.

La tracciabilità è stata trattata come parte dell'esperimento e non come documentazione accessoria. Dataset intermedi, audit, log, decisioni metodologiche e risultati finali sono versionati e separati per fase; il report utilizza gli artefatti canonici riepilogati nella [Tabella 8](#). Questa scelta consente di ricostruire l'origine di ogni risultato, distinguere le misure finali dai tentativi intermedi e conservare anche gli esiti inattesi, che in un'analisi empirica sono spesso più informativi di una sequenza di soli successi.

2. Metodologia

La metodologia è organizzata in quattro fasi concatenate. M1 produce il Dataset A e la variabile target BUGGY; M2 usa tale dataset per valutare tre classificatori e selezionare BClassifier; M3 riutilizza il classificatore selezionato per un esperimento what-if nel quale NSMELLS viene posto a zero; M4 passa dal livello storico del dataset a due classi concrete della release 3.0.0 e valuta refactoring automatici sotto disponibilità crescente di test. Questa concatenazione evita di scegliere a posteriori strumenti o dati sulla base dell'esito della fase successiva.

2.1 Milestone 1 - Dataset creation

2.1.1 Release e unità di osservazione

Il catalogo delle release è stato costruito combinando Apache Jira, GitHub Releases e tag Git locali. Jira è stato usato come fonte primaria per le release storiche, GitHub per le release pubblicate non presenti in Jira e i tag Git per associare ogni release alla revisione esatta del codice. Il catalogo completo contiene 52 release ufficiali fino alla baseline v3.0.0; l'analisi utilizza il primo 34% della sequenza cronologica, pari a 18 release, da 0.9.0.1 a 1.1.1. Questa finestra mantiene un tratto storico sufficientemente ampio per osservare l'evoluzione del sistema senza mescolare nel Dataset A le release successive usate come riferimento per le fasi finali.

Per ciascuna release viene considerato lo snapshot corrispondente al tag e vengono incluse esclusivamente classi Java classificate come PRODUCTION; test, esempi e sorgenti generati sono

esclusi. L'unità di osservazione è quindi (release, Java production class). L'inventario complessivo genera 14.611 osservazioni uniche, senza duplicati sulla chiave di release e path della classe.

2.1.2 Metriche e NSMELLS

Il Dataset A combina 24 predictor numerici, la feature NSMELLS e la label BUGGY. Le metriche non sono state raccolte come un inventario indifferenziato: rappresentano dimensioni complementari del rischio software. Le misure di dimensione e struttura descrivono il prodotto che viene mantenuto; le metriche di storia e churn catturano quanto e con quale intensità una classe cambia; quelle legate a fix, autori e distribuzione delle modifiche approssimano l'esposizione al processo di manutenzione. Questa combinazione è utile perché la defect-proneness raramente dipende da una sola proprietà: una classe grande, molto modificata e toccata da molti sviluppatori può avere un profilo di rischio diverso da una classe altrettanto grande ma stabile. La [Tabella 1](#) riassume le feature e il ruolo con cui entrano nel modeling.

Tabella 1 - Feature utilizzate nel Dataset A

Famiglia	Feature	Interpretazione
Dimensione	LOC	Linee di codice fisiche non vuote, escluse linee di solo commento.
Storia	LOC_TOUCHED, NR, NFIX, NAUTH	Volume di cambiamento, revisioni, defect fix e autori distinti.
Linee aggiunte	LOC_ADDED, MAX_LOC_ADDED, AVG_LOC_ADDED	Totale, massimo e media delle linee aggiunte.
Linee eliminate	LOC_DELETED, MAX_LOC_DELETED, AVG_LOC_DELETED	Totale, massimo e media delle linee eliminate.
Churn	CHURN, MAX_CHURN, AVG_CHURN	Totale, massimo e media del churn della classe.
Change set	CHANGE_SET_SIZE, MAX_CHANGE_SET_SIZE, AVG_CHANGE_SET_SIZE	Dimensione dei change set che coinvolgono la classe.
Età	AGE_WEEKS, WEIGHTED_AGE_WEEKS	Età della classe/release e versione pesata rispetto alle modifiche.
Dispersione	AVG_ND, MAX_ND	Numero di directory coinvolte nei change set.
Entropia	AVG_ENTROPY, MAX_ENTROPY	Entropia normalizzata della distribuzione delle modifiche.
Smell	NSMELLS	Numero di code smell rilevati sulla classe nello snapshot della release.

NSMELLS è stato misurato con due pipeline indipendenti, una basata su PMD e una sulla metrica code_smells di SonarCloud. La doppia misurazione ha un obiettivo metodologico preciso: verificare quanto il costrutto "code smell" dipenda dallo strumento che lo operationalizza e impedire che una singola tassonomia venga assunta come verità assoluta. Le due versioni del Dataset A condividono osservazioni, predictor e label e differiscono soltanto in NSMELLS; la variante SonarCloud è stata adottata come dataset principale per M2 e M3 perché fornisce una misura direttamente riconducibile ai code smell dell'analisi statica usata nel progetto, mentre la variante PMD è preservata come controllo di sensibilità del costrutto. In prospettiva predittiva, NSMELLS permette di verificare se una misura di manutenibilità aggiunge informazione alle metriche storiche e di processo; in M3 diventa inoltre la variabile manipolabile nello scenario controfattuale. Le release storiche analizzate con SonarCloud sono state eseguite sui rispettivi snapshot, ricostruendo source level, classpath e bytecode compatibili senza alterare il codice storico.

2.1.3 Identificazione dei defect, SZZ e Proportion Total

Il labeling parte dai ticket Jira classificati come defect e risolti come Fixed. Per ciascun ticket vengono individuati i fix commit associati e, sui fix non-merge, viene applicata una implementazione basic line-

based di SZZ. Il processo confronta il fix con il suo parent, abilita la rename detection, seleziona le linee preesistenti rimosse o sostituite e applica git blame sul parent per risalire ai commit che avevano introdotto quelle linee. I commit di fix appartenenti allo stesso ticket, quando ricompaiono nel blame, vengono esclusi dai candidati BIC dello stesso defect. L'analisi è limitata ai file Java production, così che l'evidenza SZZ rimanga coerente con l'unità di osservazione del dataset.

I merge vengono trattati in modo conservativo. I fix FIRST_PARENT_MERGE possono contribuire a NFIX, ma non alimentano SZZ, perché un merge può trascinare nel diff grandi quantità di codice che non rappresentano l'origine del defect. L'audit sul caso STORM-1419 ha reso concreto il rischio: includere un merge di sincronizzazione avrebbe prodotto 1.216 righe SZZ distribuite su 861 file production e 361 BIC distinti, attribuendo al ticket modifiche provenienti dal merge di apache/master. L'esclusione riduce il rischio di falsi bug-introducing changes; il prezzo è una scelta conservativa, esplicitata nelle threats to validity.

SZZ e ricostruzione temporale svolgono quindi due ruoli distinti e complementari. SZZ localizza l'evidenza del defect nel codice, mentre il lifecycle determina in quali release quella evidenza deve rendere la classe BUGGY. OV è l'ultima release ufficiale non successiva alla data di apertura del ticket; FV è la più antica Fix Version Jira valida, con Git containment usato come fallback per sei defect. Quando Jira fornisce una Affected Version ufficiale e temporalmente coerente, essa determina direttamente IV. Quando IV non è osservabile, viene stimata tramite Proportion Total, definito come $P = (FV - IV) / (FV - OV)$, ponendo il denominatore a 1 quando $FV = OV$. Il valore globale calcolato sul catalogo Jira utilizzabile è $P_TOTAL = 2,332764715904251$: sui 530 defect con evidenza SZZ, 164 usano una Affected Version per IV e 366 ricorrono alla stima proporzionale. La [Figura 1](#) sintetizza il flusso decisionale. Una volta ottenuta EffectiveIV, l'intervallo marcato buggy è [EffectiveIV, FV), con la release di fix esclusa.

SZZ e Proportion svolgono ruoli complementari: il primo localizza l'evidenza nel codice, il secondo completa la dimensione temporale quando IV non è osservabile.

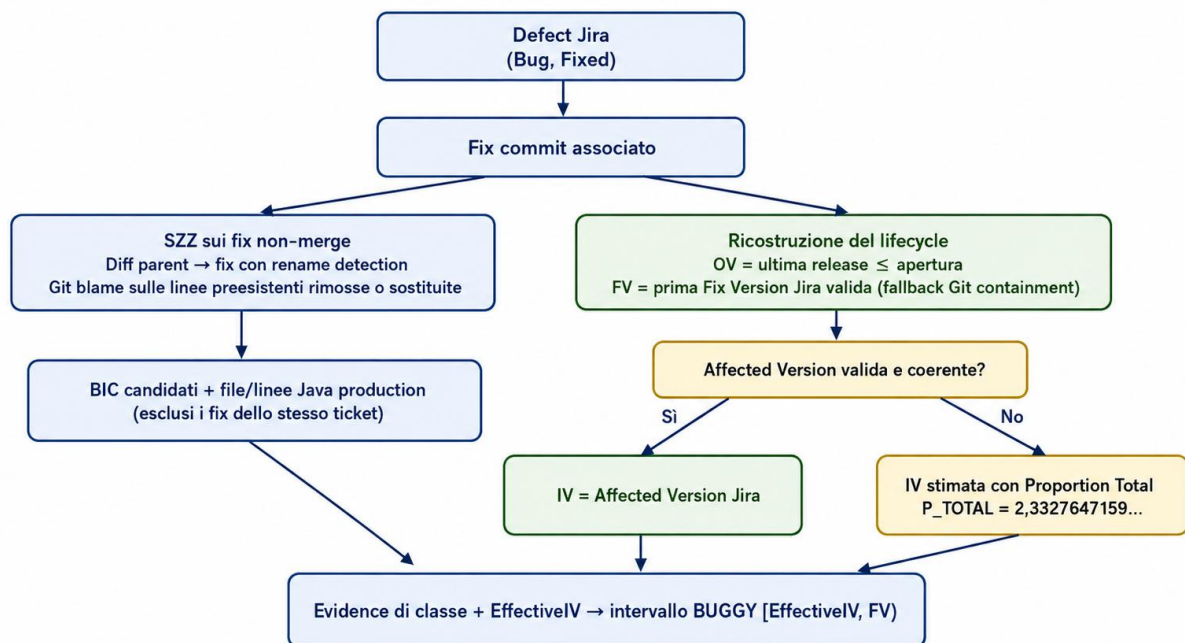


Figura 1 - Workflow di ricostruzione dell'evidenza BUGGY: SZZ e stima di IV

2.1.4 Mapping alle classi e costruzione di BUGGY

L'identità delle classi viene gestita in modo conservativo perché path e package possono cambiare tra release. Sono accettati due mapping: EXACT_BLAEMED_PATH, quando il path restituito da SZZ esiste nello snapshot analizzato, e GIT_RENAME_LINEAGE, quando la storia Git dimostra un rename univoco.

Non sono usate corrispondenze per semplice basename, similarità del path o copy detection. Una coppia (release, classe) diventa BUGGY=YES solo quando esiste almeno una evidenza sicura che la collega a un defect attivo nel relativo lifecycle. BUGGY=NO significa dunque assenza di evidenza positiva secondo questa procedura, non dimostrazione formale che la classe sia priva di defect.

2.2 Milestone 2 - Classifier accuracy

La Milestone 2 applica al Dataset A SonarCloud tre classificatori con caratteristiche differenti: RandomForest, NaiveBayes e IBk. La classe positiva è BUGGY=YES. Per ogni algoritmo vengono considerate quattro configurazioni di preprocessing: senza filtri; solo feature selection; solo balancing; feature selection e balancing. Le metriche sono Precision, Recall, AUC, Cohen's Kappa e NPofB20, così da osservare non solo l'accuratezza globale del ranking, ma anche il compromesso tra capacità di intercettare i buggy, affidabilità delle segnalazioni e utilità nelle attività di ispezione prioritaria.

La validazione è una repeated stratified 10 times 10-fold cross-validation. Le dieci ripetizioni usano seed deterministici da 1 a 10 e gli stessi fold sono condivisi da tutti i classificatori e da tutte le configurazioni. Ogni osservazione è test esattamente una volta per ripetizione e ogni riga aggregata deriva quindi da 14.611 predizioni out-of-fold. L'esperimento produce 4 configurazioni × 3 classificatori × 10 ripetizioni = 120 righe di metriche aggregate.

Per evitare data leakage, tutti i passaggi supervisionati sono confinati al training fold. La feature selection usa WEKA AttributeSelection con CfsSubsetEval + BestFirst e viene fittata sul solo training; la stessa trasformazione viene poi applicata al test. SMOTE è applicato esclusivamente al training e il test conserva la distribuzione originale. Nei 100 training fold la feature selection mantiene da 6 a 14 predictor, media 10,63; LOC, NFIX, AVG_CHANGE_SET_SIZE e AVG_ENTROPY sono selezionati in 100/100 fold, mentre NSMELLS è selezionato in 97/100.

La selezione del BClassifier non si basa su una singola metrica arbitraria. Per ognuna delle quattro configurazioni vengono identificati i vincitori delle cinque metriche, producendo 20 confronti classifier/configurazione. Il classificatore con comportamento più consistente viene portato alla Milestone 3.

2.3 Milestone 3 - What-if analysis

La Milestone 3 costruisce uno scenario controfattuale nel quale viene osservata la risposta del classificatore quando gli smell vengono rimossi a parità di tutte le altre feature. Il BClassifier selezionato in M2 è RandomForest. A partire dal Dataset A SonarCloud vengono costruiti B+ come sottoinsieme con NSMELLS > 0, C come sottoinsieme con NSMELLS = 0 e B come copia di B+ in cui solo NSMELLS viene imposto a zero. Questa costruzione isola l'effetto del predictor manipolato senza confonderlo con variazioni simultanee di LOC, churn o storia della classe.

La trasformazione B+→B è stata validata attributo per attributo: le 5.338 righe corrispondenti differiscono in esattamente 5.338 celle NSMELLS e in nessun altro predictor. RandomForest viene quindi addestrato sull'intero A con tutti i 24 predictor, senza feature selection e senza balancing, ottenendo BClassifierA. Il modello predice A, B+, B e C nascondendo BUGGY durante l'inferenza.

L'indicatore principale è definito come ActualBuggy(B+) - EstimatedBuggy(B), cioè la differenza tra le osservazioni realmente buggy nel sottoinsieme con smell e quelle che il modello continua a stimare buggy dopo l'azzeramento di NSMELLS. In parallelo vengono calcolate le transizioni paired B+→B per capire quali osservazioni cambiano previsione e in quale direzione. Le due letture sono mantenute separate: la prima misura l'effetto aggregato richiesto dall'esperimento, la seconda ne spiega la dinamica interna.

2.4 Milestone 4 - Automated refactoring

2.4.1 Selezione delle classi

La selezione parte da tutte le classi production della release 3.0.0, ordinate per NSMELLS decrescente. Per evitare che classi eccessivamente piccole rendano il refactoring poco informativo, la soglia strutturale viene fissata prima di osservare il ranking degli smell, utilizzando la distribuzione della

dimensione delle classi. In questo modo la scelta del target non viene adattata a posteriori per favorire un esito specifico.

Il candidato originario last-2, DefaultHttpCredentialsPlugin, è stato mantenuto come risultato storico della selezione. La sua suite black-box, definita prima di misurare coverage, ha però raggiunto 100% di line coverage (30/30) e 88,89% branch coverage già nella baseline T_BB, rendendo poco informativa la successiva evoluzione di testing. I test validi non sono stati rimossi per creare artificialmente un gap. È stata quindi applicata una ricerca sequenziale sullo stesso ranking congelato, senza cambiare metrica, filtro o tie-break; il primo candidato accettato dopo l'audit di testabilità è RedisFilterBolt, rank 362 (last-19), NSMELLS=0. Questa sostituzione per suitability sperimentale è documentata esplicitamente e costituisce una scelta metodologica da considerare nelle threats to validity.

2.4.2 Varianti C0-C4 e prompt

Per entrambe le classi C0 è il production code originale. Le varianti sono generate indipendentemente a partire dalla stessa C0, non in catena. C1 non riceve test; C2 riceve T_BB; C3 riceve T_BB+T_CF; C4 riceve T_BB+T_CF+T_MT. Per RedisFilterBolt non esiste una suite T_MT eseguibile aggiuntiva: il survivor residuo del mutation testing è stato classificato equivalente/ridondante, quindi C4 riceve lo stato T_MT come contesto (NO_DERIVED_SUITE) senza inventare un test mutation-guided.

Microsoft Copilot è stato istruito come sviluppatore Java esperto a migliorare la manutenibilità della classe C0, rimuovere gli smell indicati dal diagnostico SonarCloud, non cambiare la funzionalità, superare i test forniti nella specifica condizione e non introdurre modifiche estranee, poiché Cx deve poter sostituire C0 nel sistema. Ogni output è congelato prima delle misure post-hoc. Randoop, EvoSuite e le suite LLM di test generation non sono mai state fornite durante la generazione delle varianti e vengono usate soltanto come evidenze post-hoc quando eseguibili.

2.4.3 Criteri di valutazione

Per ogni Cx la valutazione combina quattro dimensioni: compilabilità della variante; presenza, rimozione o introduzione di smell; variazione delle feature positivamente correlate con la bugginess rispetto a C0; variazione delle feature negativamente correlate. Le correlazioni vengono calcolate sul Dataset A storico e poi usate come chiave di lettura delle trasformazioni: LOC e NSMELLS sono positivamente correlati con BUGGY, mentre le feature negative individuate sono prevalentemente storiche o di processo e non vengono reinterpretate artificialmente sulle varianti sintetiche.

A queste dimensioni si affianca la compatibilità comportamentale verificata tramite le suite disponibili. Il controllo è essenziale perché compilazione e riduzione degli smell non garantiscono da sole che la variante mantenga la stessa API o lo stesso comportamento osservabile. La valutazione considera quindi il refactoring come accettabile soltanto quando il miglioramento del proxy strutturale non nasconde una regressione rilevabile dai test.

3. Risultati

3.1 Milestone 1 - Caratterizzazione del dataset

La prima fotografia del dataset mostra un sistema tutt'altro che statico. Le **18 release selezionate** producono **14.611 osservazioni production**, di cui **1.243 BUGGY=YES** e 13.368 BUGGY=NO, per un buggy rate globale dell'**8,51%**. La [Tabella 2](#) permette di seguire questa evoluzione release per release e mostra subito due fenomeni: il numero di classi cambia sensibilmente nel tempo e la quota di osservazioni buggy non resta costante mentre il codebase cresce e si sposta tra linee di manutenzione differenti.

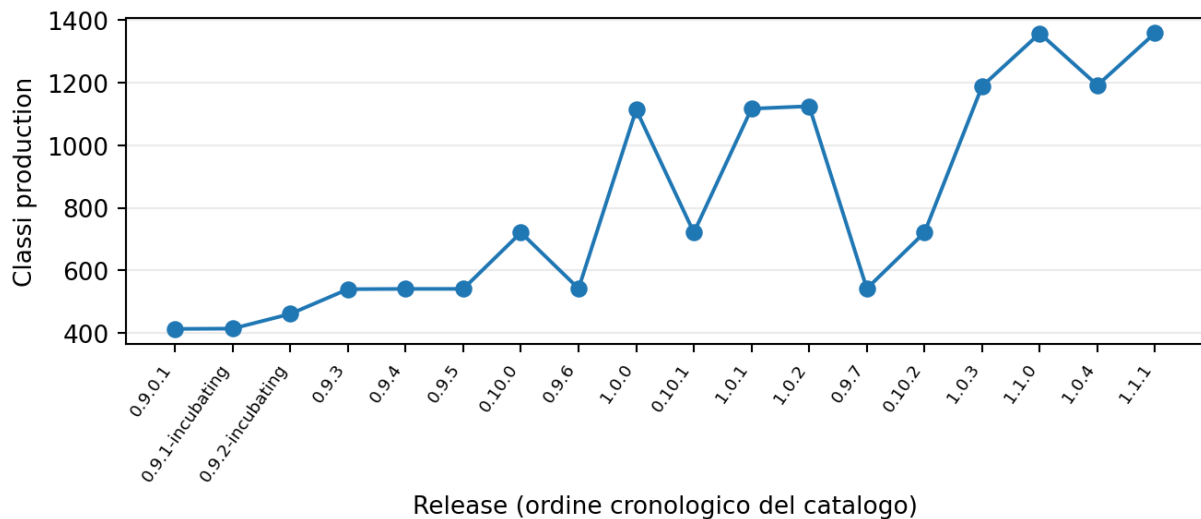
Tabella 2 - Classi production e osservazioni BUGGY per release

#	Release	Production	BUGGY	BUGGY %
1	0.9.0.1	413	21	5,08%

#	Release	Production	BUGGY	BUGGY %
2	0.9.1-incubating	414	22	5,31%
3	0.9.2-incubating	461	34	7,38%
4	0.9.3	540	44	8,15%
5	0.9.4	541	44	8,13%
6	0.9.5	541	46	8,50%
7	0.10.0	721	80	11,10%
8	0.9.6	542	55	10,15%
9	1.0.0	1114	78	7,00%
10	0.10.1	721	46	6,38%
11	1.0.1	1117	108	9,67%
12	1.0.2	1125	103	9,16%
13	0.9.7	542	32	5,90%
14	0.10.2	721	61	8,46%
15	1.0.3	1189	137	11,52%
16	1.1.0	1358	118	8,69%
17	1.0.4	1192	100	8,39%
18	1.1.1	1359	114	8,39%

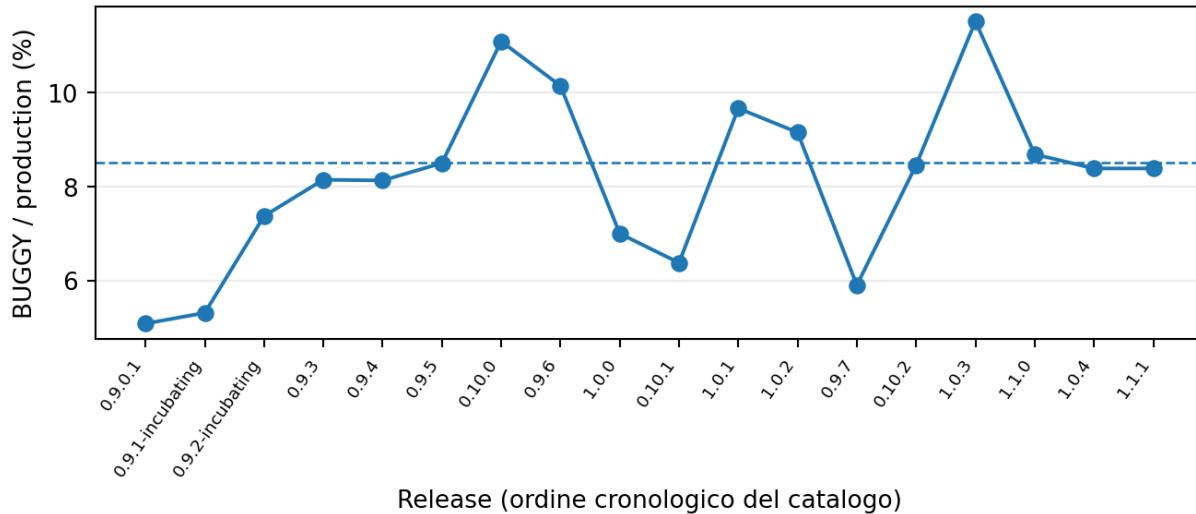
La dimensione del sistema non è stabile in senso globale: si passa da 413 classi nella 0.9.0.1 a 1.359 nella 1.1.1. La [Figura 2](#) mostra una crescita sostanziale del codebase, ma il profilo cronologico non è monotono perché il catalogo attraversa linee di manutenzione parallele 0.9.x, 0.10.x e 1.x. All'interno di alcune linee la crescita è regolare; osservando l'intero insieme, invece, la quantità di classi riflette sia l'evoluzione funzionale sia il ramo di release considerato.

Figura 2 - Numero di classi production per release



Anche la bugginess cambia con la release. Le osservazioni BUGGY sono presenti in tutte le 18 versioni, ma la loro incidenza varia dal 5,08% della 0.9.0.1 all'11,52% della 1.0.3, come mostrato in [Figura 3](#). Non emerge quindi un livello di rischio uniforme: la defect-proneness osservata dipende dalla fase evolutiva e dalla composizione del ramo. Questo risultato giustifica l'uso di metriche storiche e di processo accanto alle sole proprietà statiche del codice.

Figura 3 - Incidenza delle osservazioni BUGGY per release



Il confronto tra i due strumenti di smell evidenzia inoltre una forte dipendenza dalla definizione operativa di NSMELLS. PMD identifica 2.301 osservazioni con almeno uno smell e 5.755 smell complessivi; SonarCloud identifica 5.338 osservazioni con smell e 30.430 smell complessivi. NSMELLS differisce in 6.339 delle 14.611 osservazioni, pur mantenendo invariati tutti gli altri campi. Per questo le due misure non sono state fuse: SonarCloud è usato come Dataset A principale e PMD rimane una replica alternativa.

Il labeling finale è supportato da 3.786 righe SZZ, 530 defect con evidenza SZZ e 2.184 evidenze di mapping sicuro, che si riducono a 1.243 coppie classe-release uniche BUGGY=YES perché più defect possono colpire la stessa osservazione. Questa distinzione è importante: il dataset misura bugginess a livello di osservazione classe-release, non conta semplicemente ticket o fix commit.

Takeaway. La defect-proneness varia tra release e la misura NSMELLS dipende fortemente dallo strumento; per questo ricostruzione storica, labeling e definizione operativa degli smell sono parte integrante della validità del Dataset A.

3.2 Milestone 2 - Accuratezza dei classificatori

Il confronto tra i classificatori restituisce un quadro sorprendentemente stabile pur cambiando preprocessing e metrica. La [Tabella 3](#) riporta le medie sulle dieci ripetizioni della validazione 10×10-fold: RandomForest domina quasi tutte le configurazioni, mentre i trade-off introdotti da feature selection e balancing si riflettono soprattutto sul rapporto tra capacità di intercettare i buggy e precisione delle segnalazioni.

Tabella 3 - Risultati medi della validazione M2

Classificatore	FS	SMOTE	Precision	Recall	AUC	Kappa	NPofB20
RandomForest	No	No	0.870	0.787	0.983	0.811	0.868
RandomForest	Sì	No	0.840	0.765	0.975	0.783	0.855
RandomForest	No	Sì	0.830	0.839	0.983	0.819	0.853
RandomForest	Sì	Sì	0.767	0.819	0.974	0.772	0.843
NaiveBayes	No	No	0.273	0.316	0.767	0.222	0.194
NaiveBayes	Sì	No	0.282	0.346	0.754	0.240	0.224

Classificatore	FS	SMOTE	Precision	Recall	AUC	Kappa	NPofB20
NaiveBayes	No	Sì	0.269	0.351	0.769	0.231	0.219
NaiveBayes	Sì	Sì	0.253	0.400	0.757	0.230	0.249
IBk	No	No	0.736	0.703	0.844	0.694	0.706
IBk	Sì	No	0.758	0.762	0.934	0.738	0.791
IBk	No	Sì	0.641	0.788	0.878	0.677	0.772
IBk	Sì	Sì	0.666	0.819	0.942	0.707	0.814

RandomForest è il classificatore dominante: vince 19 dei 20 confronti ottenuti combinando quattro configurazioni di preprocessing e cinque metriche. L'unica eccezione è Recall con feature selection e SMOTE, dove IBk raggiunge 0,819308 contro 0,818504 di RandomForest, una differenza di circa 0,0008. Il miglior classificatore quindi quasi non cambia al variare della metrica/configurazione; la sola inversione è marginale e non modifica la scelta complessiva di BClassifier = RandomForest.

All'interno di RandomForest, la configurazione senza feature selection né balancing offre la Precision più alta (0,870054) e il miglior NPofB20 (0,867739). Con SMOTE ma senza feature selection si ottengono invece i valori più alti di Recall (0,838858), AUC (0,983270) e Kappa (0,818830). Il balancing produce quindi il trade-off atteso in questo dataset: recupera più osservazioni buggy, ma riduce Precision e NPofB20.

Un risultato meno intuitivo è che la feature selection non migliora RandomForest nelle metriche considerate, pur selezionando un sottoinsieme relativamente stabile dei predictor. Ciò non rende la feature selection inutile in generale: indica che, su questo dataset e con questo classificatore, la riduzione da 24 predictor a circa 10,63 in media non compensa la perdita di informazione. Analogamente, NaiveBayes rimane nettamente inferiore; IBk beneficia maggiormente di feature selection e balancing ma non raggiunge la consistenza di RandomForest.

Takeaway. RandomForest è la scelta più stabile nel confronto complessivo: domina 19/20 combinazioni metrica-configurazione, mentre SMOTE migliora soprattutto Recall, AUC e Kappa introducendo un trade-off con Precision e NPofB20.

3.3 Milestone 3 - Bug potenzialmente prevenibili

Lo scenario controfattuale produce un effetto misurabile ma non dominante. B+ contiene 5.338 osservazioni con smell, di cui 610 realmente etichettate BUGGY; C contiene 9.273 osservazioni senza smell, di cui 633 BUGGY. B conserva esattamente le 5.338 righe di B+ e modifica una sola proprietà: NSMELLS viene impostato a zero. La [Tabella 4](#) mette quindi a confronto una situazione osservata e una variante sintetica costruita per isolare il contributo operativo degli smell.

Tabella 4 - Dataset e predizioni dell'analisi what-if

Dataset	Righe	Actual BUGGY	Estimated BUGGY	Ruolo
A	14.611	1.243	1.241	Dataset originale
B+	5.338	610	609	NSMELLS > 0
B	5.338	N/A	586	B+ con NSMELLS = 0
C	9.273	633	632	NSMELLS = 0

Con l'indicatore aggregato ActualBuggy(B+) - EstimatedBuggy(B), il numero stimato di osservazioni buggy potenzialmente prevenibili è **610 - 586 = 24**. Le 24 osservazioni equivalgono al **3,9344%** delle 610 osservazioni effettivamente buggy e smelly e all'**1,9308%** di tutte le 1.243 osservazioni buggy del Dataset

A. Il valore quantifica quindi l'effetto del solo azzeramento di NSMELLS nel modello, mantenendo separate le informazioni necessarie a spiegare come le singole predizioni si spostano.

L'analisi paired offre una lettura supplementare ma non sostituisce l'indicatore aggregato: passando da B+ a B, 28 predizioni cambiano da YES a NO e 5 da NO a YES, per una riduzione netta di 23 predizioni buggy (609→586). Inoltre 29 delle 610 osservazioni di riferimento BUGGY=YES ricevono una predizione NO in B. La differenza tra 24, 23, 28 e 29 nasce da definizioni diverse; il report usa 24 come misura aggregata dell'esperimento e conserva le altre quantità come diagnostica delle transizioni.

Il risultato deve essere interpretato con cautela: B è un dataset sintetico in cui una sola feature viene modificata mantenendo tutte le altre costanti. Non dimostra che una reale attività di refactoring avrebbe evitato esattamente 24 defect. Mostra invece la risposta del modello selezionato quando il segnale NSMELLS viene eliminato in modo controllato.

Takeaway. L'azzeramento controllato di NSMELLS produce una riduzione aggregata di 24 osservazioni buggy stimate, ma l'effetto resta model-based e non causale: quantifica la risposta del classificatore, non difetti certamente prevenuti.

3.4 Milestone 4 - Esito del refactoring automatico

3.4.1 UIHelpers

UIHelpers rappresenta il caso più impegnativo del refactoring: C0 parte da LOC=1.658 e NSMELLS=77, quindi offre un margine reale di riduzione ma anche un rischio maggiore di effetti collaterali. La [Tabella 5](#) sintetizza le quattro varianti mettendo sullo stesso piano compilazione, andamento degli smell e variazione delle feature associate alla bugginess.

Tabella 5 - Risultati M4 per UIHelpers

Var.	Compila	Smell rispetto a C0	Feature positiva > C0?	Feature negativa > C0?
C1	Sì	79; +2; nessun nuovo tipo	Sì: LOC 1686; NSMELLS 79	No
C2	Sì	12; -65; solo tipi già presenti	Sì: LOC 1692	No
C3	Sì	15; -62; solo tipi già presenti	Sì: LOC 1689	No
C4	Sì	15; -62; solo tipi già presenti	Sì: LOC 1685	No

Tutte le varianti compilano, ma nessuna elimina completamente gli smell. C1 peggiora NSMELLS da 77 a 79 e aumenta da 2 a 4 le istanze della regola S5411. C2 ottiene il minimo numero di smell, 12, rimuovendo 65 istanze rispetto a C0; C3 e C4 ne mantengono 15, con una riduzione di 62. In nessun caso compare un nuovo tipo di regola Sonar rispetto a C0.

La sola riduzione degli smell non coincide però con un miglioramento univoco delle feature correlate alla bugginess. LOC, positivamente correlato con BUGGY, aumenta in tutte le varianti: +28 in C1, +34 in C2, +31 in C3 e +27 in C4. C1 peggiora inoltre NSMELLS, anch'esso positivamente correlato. Le feature negativamente correlate sono storiche/di processo e restano invariate, quindi nessuna variante mostra un aumento di tali feature.

La validazione post-hoc mette in evidenza che compilazione e Sonar non sono sufficienti a garantire behavior preservation. C1 supera T_BB, T_CF, T_RND, T_ES e T_LLM, ma fallisce 1 degli 8 test T_MT: mt06CorsConfigurationParametersAreObservable osserva null invece di "*" nella configurazione CORS. C2 riduce gli smell a 12, ma riproduce lo stesso fallimento T_MT e la suite EvoSuite congelata non compila contro C2 perché il refactoring rende privato un costruttore precedentemente accessibile. C3 è la variante più robusta nella validazione osservata: tutte le suite post-hoc eseguibili passano, incluso T_MT 8/8 ed EvoSuite 37/37. C4, nonostante abbia ricevuto anche gli 8 T_MT durante la generazione, fallisce ancora mt06 (159/160 test post-hoc passati). Le varianti non sono quindi monotonicamente migliori al crescere dell'informazione di testing.

3.4.2 RedisFilterBolt

RedisFilterBolt presenta una situazione differente: C0 ha già NSMELLS=0 e LOC=82. Di conseguenza non esistono smell da rimuovere; il criterio rilevante diventa non introdurre di nuovi e verificare se il refactoring produce altri peggioramenti. La [Tabella 6](#) sintetizza i risultati.

Tabella 6 - Risultati M4 per RedisFilterBolt

Var.	Compila	Smell rispetto a C0	Feature positiva > C0?	Feature negativa > C0?
C1	Sì	0; nessuno	No (LOC 82 = C0)	No
C2	Sì	0; nessuno	Sì: LOC 85 > 82	No
C3	Sì	0; nessuno	Sì: LOC 91 > 82	No
C4	Sì	0; nessuno	Sì: LOC 94 > 82	No

Tutte le varianti compilano e mantengono NSMELLS=0. C1 è però un no-op: il file restituito è byte-for-byte identico a C0, nonostante la risposta di Copilot descrivesse un refactoring. C2, C3 e C4 introducono modifiche strutturali reali, ma aumentano LOC rispettivamente a 85, 91 e 94. Poiché LOC è positivamente correlato con la bugginess nel Dataset A, le tre varianti non migliorano tutte le feature considerate, pur non introducendo smell.

Sul piano comportamentale, tutte e quattro le varianti superano 46/46 test eseguibili nella validazione post-hoc: 11 T_BB, 4 T_CF, 11 T_RND, 9 T_ES e 11 T_LLM. T_MT è N/A perché non esiste una suite mutation-derived eseguibile per questa classe. Il caso mostra due limiti dell'automated refactoring: senza test il modello può non modificare affatto il codice; con più contesto può produrre cambiamenti compatibili con le suite ma non necessariamente migliori rispetto a tutte le feature correlate alla bugginess.

3.4.3 Lettura complessiva delle varianti

Il confronto tra le due classi mostra che “meno smell” e “migliore refactoring” non sono sinonimi. UIHelpers consente una forte riduzione degli smell con C2-C4, ma nessuna variante raggiunge zero e le trasformazioni introducono costi o regressioni che emergono soltanto quando il codice viene osservato oltre il conteggio Sonar. RedisFilterBolt parte invece già da NSMELLS=0: in questo caso il valore dell'esperimento non sta nella rimozione di smell, ma nella capacità del refactoring di evitare degradazioni su un codice che non presentava il problema target.

La quantità crescente di test fornita al modello agisce come vincolo, ma non genera una progressione monotona. Su UIHelpers C2 ottiene il minimo NSMELLS, mentre C3 è la variante che preserva meglio le suite post-hoc; C4 riceve anche T_MT ma mantiene una regressione CORS. Su RedisFilterBolt C2-C4 restano compatibili con le suite, ma aumentano progressivamente LOC. Il dato più rilevante è quindi il trade-off: i test aiutano a rendere osservabili alcuni errori, ma non trasformano automaticamente una ottimizzazione di smell in un miglioramento complessivo del software.

Takeaway. Più contesto di testing non produce refactoring monotonicamente migliori: smell, feature associate alla bugginess e behavior preservation devono essere letti insieme, perché una variante può migliorare su una dimensione e peggiorare su un'altra.

4. Threats to validity

I risultati vanno letti alla luce delle decisioni necessarie per trasformare una storia reale di progetto in un esperimento riproducibile. Ogni fase introduce un possibile scarto tra il costrutto che si vuole osservare e la sua misura operativa: BUGGY dipende dalla qualità di Jira, SZZ e mapping; NSMELLS dalla tassonomia dello strumento; le prestazioni predittive dal protocollo di validazione; il what-if dalla plausibilità della trasformazione sintetica; il refactoring automatico dalla variabilità del modello e dai test

disponibili. La [Tabella 7](#) riassume per ciascuna minaccia la conseguenza possibile, la mitigazione adottata e il limite che rimane aperto.

Tabella 7 - Threats to validity, conseguenze e mitigazioni

Minaccia	Possibile conseguenza	Mitigazione adottata	Limite residuo
External validity	Un solo progetto, prime 18/52 release e due classi M4: i risultati possono non generalizzare ad altri sistemi o periodi.	Dataset storico ampio e procedure deterministiche; conclusioni formulate come case study su Storm.	Repliche su altri progetti e classi restano necessarie.
Construct - BUGGY	Jira, mapping dei fix e SZZ sono proxy imperfetti; BUGGY=NO può includere defect non ricostruiti.	SZZ conservativo, mapping rename-aware, evidence auditabili; nessun mapping euristico per basename.	Possibili falsi negativi/positivi residui.
Construct - NSMELLS	Il numero di smell dipende dallo strumento e dalle regole.	PMD e SonarCloud mantenuti separati; SonarCloud scelto come dataset principale; differenze documentate.	6.339 osservazioni hanno NSMELLS diverso tra strumenti; 7 parse error SonarJava.
Internal - SZZ merge	Merge di sincronizzazione possono produrre BIC spurii.	Esclusione FIRST_PARENT_MERGE dopo audit STORM-1419; 133 fix dello stesso issue esclusi dal blame.	L'approccio conservativo può perdere alcune evidenze valide.
Internal - Proportion	IV stimata per defect privi di Affected Version può spostare l'intervallo buggy.	P_TOTAL calcolato dinamicamente su 301 defect utilizzabili; formula e invarianti validate.	366/530 lifecycle SZZ usano Proportion Total.
Internal - M2 leakage	Feature selection o SMOTE applicati prima dello split gonfierebbero le metriche.	Entrambi fittati nel training fold; SMOTE mai sul test; fold identici tra configurazioni.	Le 10 ripetizioni riutilizzano lo stesso dataset, quindi non sono campioni indipendenti.
Internal - M3 counterfactual	Porre solo NSMELLS=0 non simula tutti gli effetti reali del refactoring.	B differisce da B+ in una sola feature e viene validato riga-per-riga; risultato dichiarato model-based.	Nessuna inferenza causale: 24 è una stima del modello.
Internal - M4 LLM	Output Copilot non deterministico e potenzialmente sensibile al prompt.	Prompt, input, output e SHA congelati; varianti generate indipendentemente da C0 e non corrette post-hoc.	Una singola generazione per condizione non stima la distribuzione di possibili output.
Selection - M4 seconda classe	RedisFilterBolt sostituisce il last-2 originario per suitability; la scelta può introdurre bias.	Ranking/filtro originali congelati; scansione sequenziale; evidence del candidato rifiutato preservate; nessun test valido rimosso.	La classe finale non coincide con la posizione letterale last-2 del protocollo iniziale.
Construct - M4 history	Metriche storiche non sono ridefinibili in modo naturale per file sintetici Cx.	Feature storiche/process restano invariate; si rimisurano solo proprietà statiche applicabili.	L'analisi non cattura una futura storia reale della variante.
Conclusion validity	Differenze piccole o trade-off possono essere sovrainterpretati.	M2 usa 10×10-fold e riporta medie; M3 separa 24 ufficiale da 23/28/29 diagnostici; M4 conserva regressioni.	Non sono effettuate rivendicazioni causali o universali.

La minaccia più importante per M1 riguarda il construct BUGGY. Il workflow è volutamente evidence-driven: l'assenza di mapping non viene trasformata in evidenza positiva e BUGGY=NO non è presentato come certificazione di assenza di defect. Il costo è una maggiore probabilità di falsi negativi rispetto a strategie aggressive, ma la scelta riduce associazioni arbitrarie tra classi diverse.

Per M2 la principale minaccia evitabile era il leakage. Poiché feature selection e SMOTE utilizzano informazione sul target, applicarli sull'intero dataset prima della cross-validation avrebbe contaminato il test. Il progetto li applica invece all'interno del training fold. Questo rende più credibile il confronto tra le 12 combinazioni, pur non eliminando il limite esterno di lavorare su un singolo dataset di progetto.

M3 è per definizione controfattuale. La trasformazione isolata di NSMELLS è utile perché controllata e riproducibile, ma un vero refactoring potrebbe modificare LOC, churn, complexity, change-set e altre proprietà. Il numero 24 non va quindi comunicato come numero di defect realmente evitabili, bensì come stima di BClassifierA nello scenario modellato.

M4 mostra concretamente il rischio di affidarsi a proxy singoli. C2 di UIHelpers ottiene il minimo NSMELLS ma modifica l'accessibilità del costruttore e rende incompatibile la suite EvoSuite congelata; C4 riceve anche T_MT ma mantiene una regressione CORS osservabile. Per evitare survivorship bias questi esiti sono stati mantenuti nel dataset sperimentale e le varianti non sono state corrette dopo la misura. La sostituzione della seconda classe, infine, è una deviazione dichiarata dal last-2 originario: è stata adottata per evitare una baseline di testing già satura, ma limita la stretta aderenza alla selezione letterale e viene trattata come threat, non nascosta.

5. Conclusioni

Il valore principale del progetto non è un singolo numero, ma la continuità empirica costruita tra storia, previsione, scenario controfattuale e trasformazione del codice. M1 ricostruisce 14.611 osservazioni classe-release su 18 release e mostra che la defect-proneness di Apache Storm è un fenomeno dinamico: il dataset contiene 1.243 osservazioni BUGGY (8,51%), ma la loro incidenza cambia insieme alla struttura e alla storia del sistema. La forte differenza tra PMD e SonarCloud rende inoltre evidente che uno smell non è una proprietà assoluta del software, ma un costrutto operativo che dipende dalle regole con cui viene misurato.

Su questa base, M2 produce un risultato robusto: RandomForest vince 19 dei 20 confronti metrica/configurazione. Il dettaglio è più interessante della sola classifica, perché il preprocessing cambia il tipo di prestazione che viene privilegiata. La configurazione senza balancing massimizza Precision e NPofB20; SMOTE, senza feature selection, spinge invece Recall, AUC e Kappa. La stabilità del classificatore e la variabilità del preprocessing suggeriscono una distinzione utile anche operativamente: la scelta del modello può essere relativamente stabile, mentre la soglia di attenzione verso falsi positivi o falsi negativi deve dipendere dall'obiettivo di manutenzione.

M3 quantifica ciò che il solo confronto statistico non avrebbe mostrato: azzerare NSMELLS a parità delle altre feature riduce di 24 le osservazioni buggy stimate secondo l'indicatore aggregato, pari al 3,9344% dei buggy con smell e all'1,9308% di tutti i buggy di A. L'effetto è abbastanza grande da rendere gli smell rilevanti come segnale di qualità, ma abbastanza contenuto da escludere una lettura semplicistica secondo cui "rimuovere gli smell" significhi eliminare la maggior parte del rischio. La defect-proneness resta il risultato di più dimensioni, soprattutto storiche e di processo.

M4 porta questa conclusione dal modello al codice e ne mostra la conseguenza più concreta. Su UIHelpers gli smell scendono da 77 a 12-15 nelle varianti C2-C4, ma non arrivano a zero; tutte le varianti aumentano LOC e alcune alterano compatibilità o comportamento. Su RedisFilterBolt la baseline possiede già zero smell: C1 non modifica il codice e C2-C4 preservano i test, ma aumentano LOC. Il risultato più utile non è quindi stabilire quale Cx "vince", bensì osservare che il refactoring automatico ottimizza proxy locali e deve essere governato da vincoli comportamentali, metriche multiple e revisione delle trasformazioni. Più test aumentano l'informazione disponibile, ma non garantiscono una graduatoria monotona della qualità.

5.1 Impatto per i practitioner

Per i practitioner, lo studio suggerisce un modo concreto di integrare analytics e manutenzione. Gli smell possono contribuire alla prioritizzazione, ma acquistano valore quando vengono letti insieme a churn, storia dei fix, ownership e dimensione della classe. Un modello di defect prediction può quindi essere usato come strumento di triage per concentrare review e testing sulle aree più esposte, non come classificatore infallibile. Allo stesso modo, un refactoring automatico non dovrebbe essere accettato perché riduce un contatore Sonar: deve compilare, preservare il comportamento e non peggiorare altre

proprietà associate al rischio. L'esperimento su UIHelpers mostra esattamente perché questa disciplina è necessaria.

5.2 Impatto per i researcher

Per i researcher, il case study evidenzia quanto i risultati empirici dipendano dalla definizione dei construct e dalla tracciabilità delle pipeline. NSMELLS cambia sensibilmente tra strumenti; BUGGY deriva dall'interazione tra issue tracker, SZZ, lifecycle e mapping; il what-if dipende dalla semantica della manipolazione; l'automated refactoring dipende dal modello generativo e dal contesto di test. La replicabilità richiede quindi non soltanto codice, ma evidence delle decisioni, risultati intermedi e regole di esclusione. Il progetto conserva questi elementi proprio per permettere di distinguere un effetto stabile da un artefatto del processo sperimentale.

La prosecuzione naturale è estendere la stessa catena a più progetti, classi e generazioni LLM, ma il contributo del caso Storm è già chiaro: non dimostra che “zero smell” equivalga a “zero bug”, e proprio per questo offre un risultato più utile. Mostra che la qualità software emerge dall'interazione tra struttura, storia, processo e verifiche comportamentali; che gli smell possono essere un segnale azionabile ma non autosufficiente; e che l'automazione diventa realmente utile quando viene inserita in una pipeline misurabile, tracciabile e capace di rifiutare anche trasformazioni apparentemente migliori. È questa continuità tra evidenza storica e decisione di manutenzione a rendere il progetto rilevante oltre il singolo dataset.

Riferimenti e tracciabilità

L'impostazione metodologica e i criteri di valutazione derivano dal materiale ufficiale del corso. I risultati quantitativi riportati nel documento provengono dagli artefatti versionati del repository; la [Tabella 8](#) raccoglie i source of truth principali utilizzati nella stesura, così da rendere ogni valore verificabile rispetto alla pipeline che lo ha prodotto.

Materiale didattico primario

- Falessi, D. (2026). Final Project.
- Falessi, D. (2026). Milestone 1: Dataset Creation.
- Falessi, D. (2026). Milestone 2: Classifiers Accuracy.
- Falessi, D. (2026). Milestone 3: How many buggly classes could have been prevented by having zero smells?
- Falessi, D. (2026). Milestone 4: Characterizing Automated Refactoring.

Artefatti canonici di progetto

Tabella 8 - Artefatti canonici usati per il report

Fase	Artefatto
M1 - Dataset A SonarCloud	isw2/datasets/storm_m1_dataset_sonarcloud.csv
M1 - metodologia dataset	isw2/docs/m1-dataset.md
M1 - lifecycle / Proportion	isw2/docs/defect-lifecycle.md
M1 - SZZ	isw2/docs/szz-analysis.md
M1 - BUGGY labeling	isw2/docs/buggy-labeling.md
M2 - metodologia e risultati	isw2/docs/m2-classifiers.md
M2 - classificatore selezionato	isw2/results/m2/best_classifier.csv
M3 - metodologia	isw2/docs/m3-what-if.md
M3 - summary	isw2/results/m3/what_if_summary.csv
M4 - selezione classi	isw2/docs/testing-class-selection.md
M4 - UIHelpers final analysis	isw2/results/m4/uihelpers/uihelpers-m4-analysis.md
M4 - RedisFilterBolt final analysis	isw2/results/m4/redisfilterbolt/redisfilterbolt-m4-analysis.md

Repository: <https://github.com/BestOfTorac/storm> - branch isw2-project